

function

function is a block of code, which has some name for identification

```
functionName ( )
```

```
{
```

```
    _____ ;
```

```
    _____ ;
```

```
    _____ ;
```

```
    _____ ;
```

```
    _____ ;
```

```
}
```

function
Definition

functions are of two types

Predefined functions

Userdefined functions

func() definition is stored in c-library

★

func() will execute only when it is called

Example :-

```
main()  
{  
    a();  
    b();  
    a();  
}  
a()  
{  
    printf("Hello");  
}  
b()  
{  
    printf("Bye");  
    a();  
}
```

- In this program 3 function are defined

function called in this program -

```
a()  
b()  
printf()
```

But one function which is not called by us but it is called by operating system is `main()`. for the execution of program

Whenever a function is called firstly it gets memory in ram

As soon as function works complete its memory get release

And control get back to the position from where the function is called particular

Ways to define a function

1. Takes Nothing, Returns nothing TNRN
2. Takes something, Returns nothing TSRN
3. Takes Nothing, Return something TNRS
4. Takes something, Return something TSRS

In C - file there is function definition

In header - file there is function declaration
pre-defined

#include <stdio.h> → Pre-defined function declaration

void add(); → User-defined function declaration

```
int main()
```

```
{
```

```
    add(); → function call
```

```
    return 0;
```

```
}
```

} function definition

```
void add() → Empty parenthesis means TN
```

```
{
```

return type

Takes nothing

```
    int a, b, sum = 0;
```

```
    printf("Enter the value of a and b = ");
```

```
    scanf("%d %d", &a, &b);
```

```
    sum = a + b;
```

```
    printf("sum = %d", sum);
```

```
}
```

No return keyword means RN

return nothing

} function definition

In front of function name we have to define return type
If the function is of RN (return nothing) type then we use void as return type

Take something return nothing
x x x

```
#include <stdio.h>
void add (int ,int )
```

```
int main ()
{
```

```
    int x, y;
    printf ("Enter the value of x and y = ");
    scanf ("%d %d", &x, &y);
    add(x, y) → Actual argument → TS
    return 0;
```

```
}
```

```
void add (int a, int b) → formal argument
```

```
{
```

```
    int c;
    c = a + b;
    printf ("Sum is %d", c);
```

```
}
```


Take nothing Return something

```
#include <stdio.h>
```

```
int add()
```

```
int main()
```

```
{
```

```
    int s;
```

```
    s = add();
```

```
    printf("Sum is %d", s);
```

```
    return 0;
```

```
}
```

```
int add()
```

```
{
```

```
    int a, b, c;
```

```
    printf("Enter two numbers");
```

```
    scanf("%d %d", &a, &b);
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

When the control comes at return then the control get back to the function (add()) from where it is called so,

value of c gets assign in s

Take something return something

```
#include <stdio.h>
```

```
int add (int, int);
```

```
int main ()
```

```
{
```

```
    int a, y, s;
```

```
    printf("Enter two numbers = ");
```

```
    scanf("%d %d", &a, &y);
```

```
    s = add(a, y);
```

```
    printf("sum is %d", s);
```

```
    return 0;
```

```
}
```

```
int add (int a, int b)
```

```
{
```

```
    int c;
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

*

Jb hum kuch ~~Karwa~~ print karwa rhe hote hai tu Return nothing hota hai